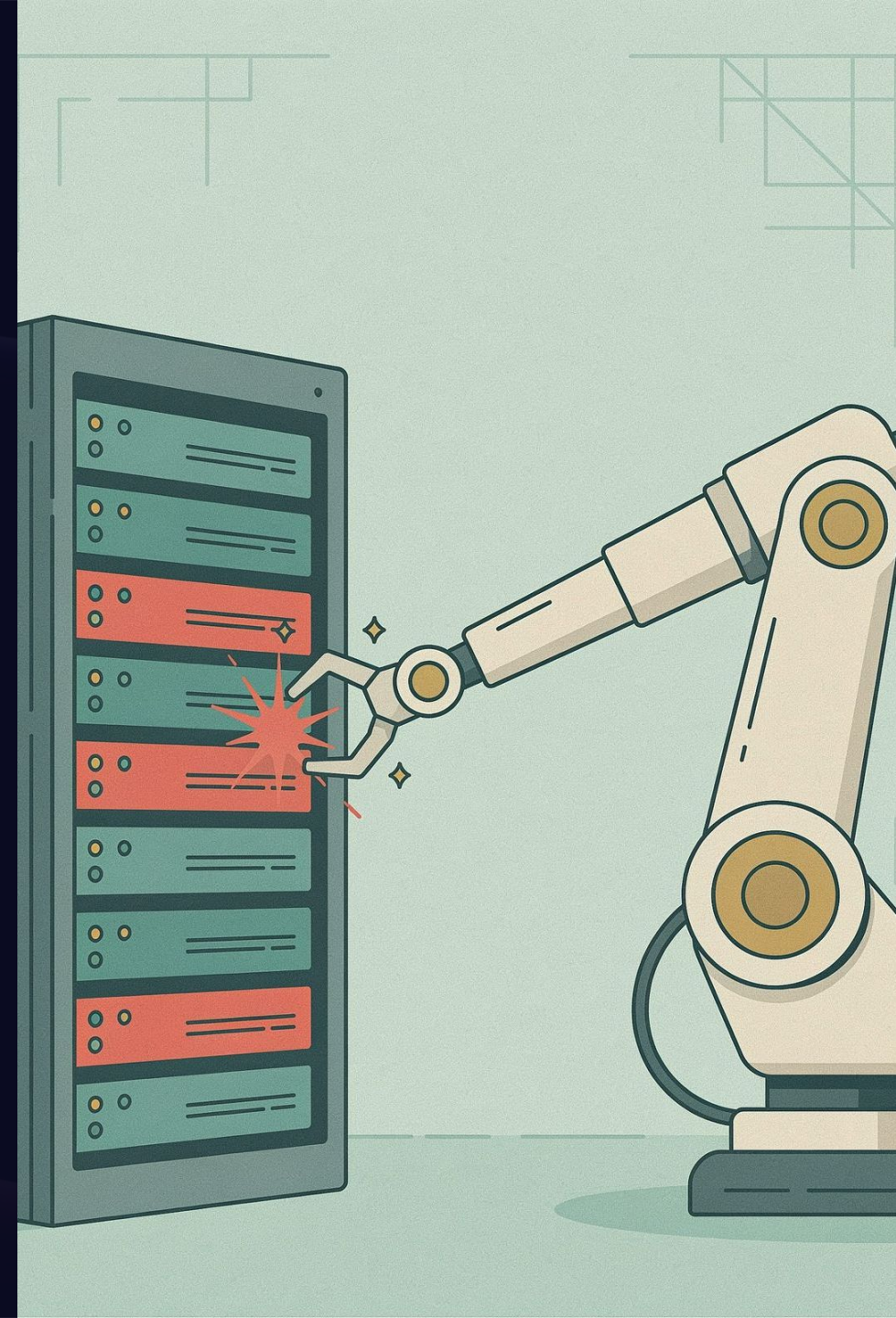


# Building Enterprise Agents with Python and Tool Calling

Ramesh Sannareddy

ENTERPRISE AI ENGINEERING





LLM Decision

Tool Call

Enterprise System

Result Returned

## Building Agents That Can Act

A language model alone can reason, summarize, and explain — but it cannot change the state of the world. An agent becomes genuinely useful only when it can reach beyond the model boundary and interact with external systems. Tool calling is the mechanism that closes that gap: the LLM decides what to do, and Python controls whether and how it happens.

□ An agent becomes useful when it can interact with the world outside the model.



# The Problem with a Standalone LLM

Ask Claude: "**Can you onboard an employee?**" Technically — no. Claude can describe the process in detail, but it cannot act on it. Without tool access, a language model is a sophisticated text generator, not an agent.

## LLM Alone

User → LLM → Text

- Can describe steps
- Cannot query HR database
- Cannot check laptop inventory
- Cannot schedule a meeting
- Cannot create an IT ticket

## Agent with Tools

User → Agent → LLM → Tools → Enterprise Systems

- Queries HR database
- Checks equipment inventory
- Schedules induction sessions
- Creates IT tickets automatically
- Updates enterprise applications

# What Is a Tool?

A tool is a controlled capability exposed to an agent through a well-defined interface. Crucially, a tool does not need to be an AI capability — it can be any ordinary function your application already has. The key is that the interface is explicit, predictable, and controlled.

## **get\_employee()**

Retrieve employee profile by ID

## **check\_equipment()**

Verify laptop and hardware availability

## **get\_access\_requirements()**

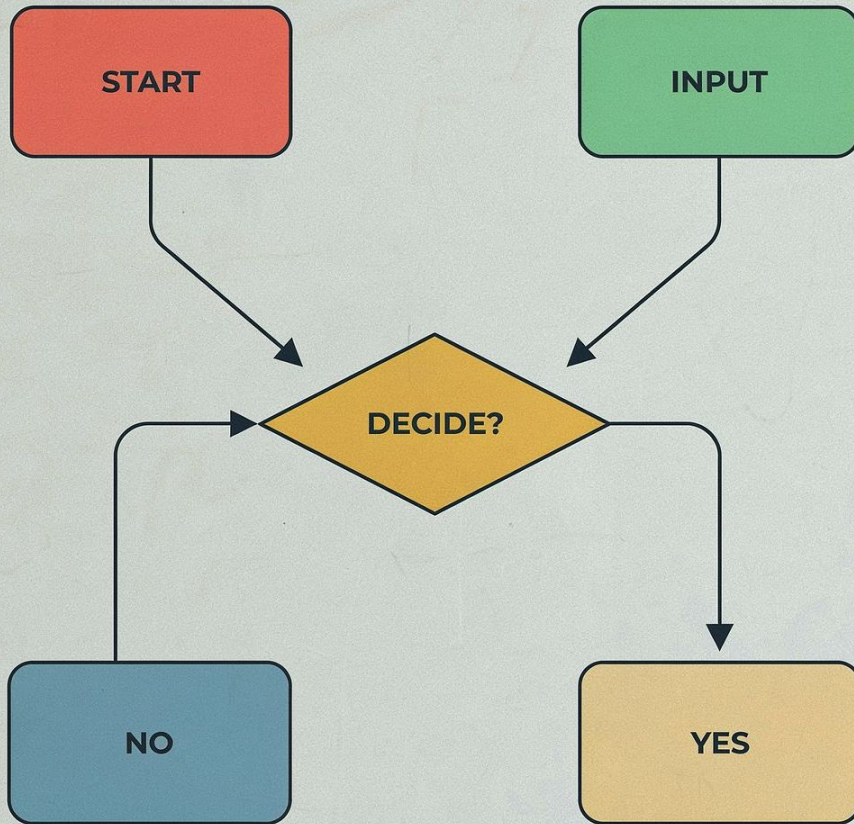
Look up system access by role

## **create\_ticket()**

Open an IT provisioning ticket



Tools are ordinary software capabilities that the LLM can request through a controlled interface — nothing more, nothing less.



# Tool Calling: The Big Picture

User Request

Agent Receives

LLM Reasons

Tool Selection



The model does not execute the function. The application executes it. This distinction is foundational — repeat it throughout your implementation.

# The Tool-Calling Protocol

Tool calling follows a precise six-step protocol. Understanding each step is essential for building reliable enterprise agents. The loop — LLM → Tool Request → Python → Tool → Result → LLM — is the architectural heartbeat of every tool-using agent.

## 1 Application sends context

System instructions, the user's request, and the full list of available tools are passed to the LLM in a single call.

## 2 LLM responds with a tool request

The model returns a structured response: "Call tool X with arguments Y." It does not call anything itself.

## 3 Application validates and executes

Python validates the request against the registry, executes the tool, and sends the result back to the LLM for its next decision.

# A Tool Is an API Contract

For senior engineers, the most useful conceptual anchor is this: **tool design is API design**. Every tool has a name, a typed input signature, and a predictable output structure. The LLM reasons about this contract — it does not see implementation details.

## The Function Signature

```
def get_employee(  
    employee_id: str  
) -> dict:  
    ...
```

## Example Output

```
{  
    "employee_id": "EMP-1042",  
    "name": "Priya",  
    "department": "Engineering",  
    "role": "Senior Data Engineer"  
}
```

The contract defines three things: **what the tool is called**, **what it accepts as input**, and **what it reliably returns**. Treat every tool as a public API surface — design it accordingly.

# Tool Schema Design

The LLM cannot call a tool it doesn't understand. The schema you provide tells the model what a tool does, when to use it, and what parameters it requires. **Descriptions matter as much as types.**

## Schema Structure

```
{
  "name": "get_employee",
  "description": "Retrieve employee
  profile information using the
  employee ID. Use this when employee
  identity or department info
  is required.",
  "parameters": {
    "type": "object",
    "properties": {
      "employee_id": {
        "type": "string"
      }
    }
  },
  "required": ["employee_id"]
}
```

## Poor vs. Better Description

### Poor:

"Gets employee."

### Better:

"Retrieve employee profile  
information using the employee  
ID. Use this **when** employee  
identity or department  
information is required."

📌 The description is the LLM's only guide for deciding when and how to use the tool. Invest in it.

# Tool Selection

When the agent receives a goal, the LLM must reason about *which tool to use first* — and in what order. Given the available set below, a request to onboard EMP-1042 produces a structured decision, not freeform text.

get\_employee

check\_equipment

get\_access\_requirements

find\_induction\_slots

The LLM's response for "**Onboard EMP-1042**" is not a sentence — it is a structured tool request:

```
{ "tool": "get_employee", "arguments": { "employee_id": "EMP-1042" } }
```

📌 Tool calling introduces decision-making about actions, not just text generation. The model must reason about state and sequence.

# Tool Results Become Observations

After each tool executes, its result is returned to the LLM as an observation. The model reads the new information and decides what to do next — call another tool, ask a clarifying question, or produce a final response. This is the core of the agent reasoning loop.

## Tool Call

```
get_employee("EMP-1042")
```

## Returns

```
{  
  "name": "Priya",  
  "department": "Engineering",  
  "role": "Senior Data Engineer"  
}
```

## The Loop

01

---

### Decision

LLM determines next action

02

---

### Tool Call

Python executes the function

03

---

### Observation

Result fed back to LLM

04

---

### Decision

LLM reasons from new state



# Employee Onboarding: Multi-Tool Workflow

A single user request — **"Onboard EMP-1042"** — can trigger a chain of tool calls. Each call produces an observation that informs the next decision. This is what separates an agent from a chatbot.

1

**get\_employee()**

Name: Priya

Role: Senior Data Engineer

2

**get\_access\_requirements()**

Required: GitHub, Jira, AWS

3

**check\_equipment()**

Laptop: Available

4

**find\_induction\_slots()**

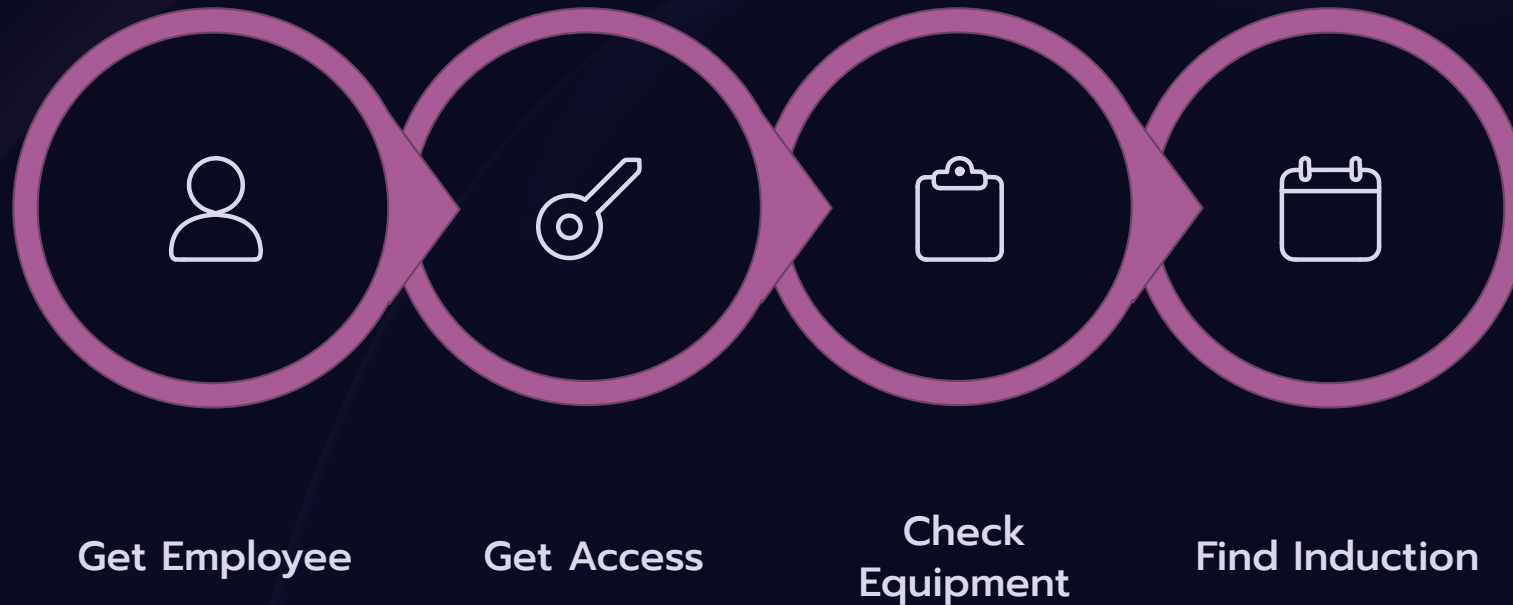
Monday 10 AM available



One user request can result in many tool calls. The agent orchestrates the sequence automatically.

# Sequential Tool Calling

Many enterprise workflows involve dependencies between steps. The agent cannot determine which access systems to provision until it knows the employee's role. This requires sequential execution — each tool call depends on the observation from the previous one.



Dependencies matter. Enforcing sequencing correctly — and recognizing when it is not required — is a key engineering concern in multi-tool agent design.

# Parallel Tool Calling

Not every tool call must wait for the previous one. Once the agent knows the employee's department and role, it may be able to check equipment availability and retrieve access requirements simultaneously — reducing total latency.

## Known State

- Department: Engineering
- Role: Senior Data Engineer

## Parallel Calls

- `check_equipment()`
- `get_access_requirements()`

Both calls are independent — they can execute concurrently.

## Engineering Trade-offs

### Benefit

Reduced end-to-end latency for workflows with independent steps

### Cost

Additional complexity in result aggregation, error handling, and partial failure scenarios

### Rule

Parallelize only when results are truly independent and you can handle concurrent failures cleanly

# The Tool Registry

Don't scatter tool definitions throughout your application. Centralize them in a registry — a simple dictionary mapping names to callables. The registry is the single source of truth for what the agent is allowed to do.

## Registry Pattern

```
tools = {  
    "get_employee":  
        get_employee,  
    "check_equipment":  
        check_equipment,  
    "get_access_requirements":  
        get_access_requirements,  
    "find_induction_slots":  
        find_induction_slots,  
}
```

## Why a Registry?

- **Discoverability**

All capabilities in one place

- **Validation**

Reject unknown tool names before execution

- **Centralized control**

Enable or disable tools in one location

- **Logging**

Audit every tool invocation

- **Security**

The agent can only call what you expose

# Tool **Validation**: Never Trust LLM Output

This is one of the most important production lessons in the course. The LLM controls the tool name in its response — and LLM output is untrusted input. Executing it directly is a serious security vulnerability.

## ✗ Never Do This

```
# Dangerous: LLM controls the name
tool = globals()[tool_name]
tool(**arguments)
```

An adversarial or hallucinating model could invoke any function in your Python runtime.

## ✓ Always Validate First

```
if tool_name not in tool_registry:
    raise ValueError(
        "Tool not allowed"
    )

tool = tool_registry[tool_name]
result = tool(**arguments)
```

Only registered, explicitly allowed tools can be invoked — nothing else.

⊗ LLM output is untrusted input. Treat every tool request from the model as you would treat input from an external user — validate it before acting.

# Validate Tool Arguments

Schema validity is not the same as business validity. Even if the LLM provides a value of the correct JSON type, that value may violate your domain rules. A robust validation pipeline catches problems before execution, not after.



📌 **Schema validity ≠ business validity.** A model passing `equipment_type: 12345` when a string is expected will fail schema validation — but a valid string like `"unknown_type"` still needs business-level checks.

# Read Tools vs. Action Tools

Before building Lab 3, establish this critical architectural distinction. Read-only tools retrieve information without changing system state. Action tools modify the enterprise environment — and that difference has significant implications for authorization, auditing, and human oversight.

## Read-Only Tools

### **get\_employee**

Retrieve profile data

### **check\_equipment**

Check availability

### **get\_policy**

Read access rules

### **find\_induction\_slots**

View available times

Low risk. Returns information only. No side effects.

## Action Tools

### **reserve\_equipment**

Allocates a physical asset

### **schedule\_induction**

Books calendar time

### **create\_ticket**

Opens an IT request

### **send\_email**

External communication

Higher risk. Changes enterprise state. Requires stronger authorization and audit trail.

# Tool Calling **Failure Modes**

Production agents fail in predictable ways. Design for these scenarios explicitly — don't assume the happy path is the only path.



## Wrong Tool

Agent selects an inappropriate capability for the current context



## Wrong Arguments

Missing, malformed, or out-of-range parameters passed to the tool



## Tool Failure

Downstream API unavailable or returning unexpected errors



## Infinite Loop

Agent repeatedly calls the same tool without making progress



## Tool Hallucination

Model requests a tool that does not exist in the registry



## Over-Permission

Agent has access to capabilities it should never need for its task

# Designing Safe Tool Boundaries

Apply the principle of least privilege to tool design. The more capabilities an agent has, the larger the blast radius of a failure — whether from a hallucination, an adversarial input, or a logic error in the model's reasoning.

## ✗ Avoid: Broad Access

Agent → Entire Enterprise API

Any failure or compromise can affect any system the API exposes.

## ✓ Prefer: Narrow Scope

Agent

- └── get\_employee
- └── check\_equipment
- └── get\_access\_requirements
- └── find\_induction\_slots

Minimal surface area. Clear boundaries.

## Tool Safety Checklist

- **Narrow scope**  
Each tool does exactly one thing
- **Clear inputs**  
Typed, validated, constrained parameters
- **Predictable outputs**  
Consistent schema the LLM can reason about
- **Explicit authorization**  
Not every agent gets every tool
- **Controlled side effects**  
Action tools require audit trail and human gates

# The Python Agent Loop

Here is the core architecture in its simplest form. Every production tool-calling agent — regardless of framework — is a variation of this loop. If you understand this, you understand the foundation.

## The Core Loop

```
while True:
    response = llm.generate(
        messages=messages,
        tools=tools
    )

    if response.is_final():
        break

    tool_call = response.tool_call
    validate_tool_call(tool_call)
    result = execute_tool(tool_call)

    messages.append(response)
    messages.append(result)
```

## What Each Line Does

01

---

### Generate

LLM reasons over current message history and available tools

02

---

### Check terminal

If the model has a final answer, exit the loop

03

---

### Validate

Check tool name against registry and validate arguments

04

---

### Execute

Python runs the function and captures the result

05

---

### Append and repeat

Result becomes the next observation; loop continues

# From Tool Calling to an Enterprise Agent

Bringing it all together: the LLM proposes the next capability, and Python controls whether and how that capability executes. Every component in the architecture below exists to make that boundary safe, auditable, and reliable.



□ **The LLM chooses the next capability. Python controls whether and how that capability is executed.** This is the architectural principle that makes enterprise agents safe to deploy.

# Lab 2: Give Your Agent One Capability

Your agent currently knows how to reason. Now give it the ability to act. Start with a single tool, then expand the workflow step by step — this is how real enterprise agents are built.

1

## 1 Tool

`get_employee("EMP-1042")` — agent understands goal, selects tool, receives result, uses it in response

2

## 3 Tools

Add `check_equipment` and `get_access_requirements` — agent sequences calls based on observations

3

## Multi-step workflow

Full onboarding sequence: `employee` → `access` → `equipment` → `induction` → `tickets`

4

## Human approval gates

Action tools require confirmation before execution — setting up Lab 3

5

## Enterprise agent

Production-grade: registry, validation, authorization, audit logging, error handling

# Three Things to Remember

These three principles are the conceptual foundation for everything in Lab 3. Carry them forward.

## 1. A tool is an API contract

Name + description + typed schema + controlled execution. Tool design is API design — invest in your schemas and descriptions.

## 2. Tool calling is a loop

LLM → Tool Request → Python → Tool  
→ Observation → LLM. Every agent framework is a variation of this loop.

## 3. The LLM is not the security boundary

The model proposes. The application validates, authorizes, and executes. This distinction sets up Lab 3's guardrails and human approval architecture.

📌 If you wish to pick only one idea from this session, make it the third one: the application — not the model — is the security boundary.